

Esercizio 1. Riscrivete le procedure ENQUEUE e DEQUEUE per una coda viste a lezione in modo da rilevare underflow e overflow.

ENQUEUE(Q, x)

→
1 $Q[Q.tail] = x$
2 **if** $Q.tail == Q.size$
3 $Q.tail = 1$
4 **else** $Q.tail = Q.tail + 1$

DEQUEUE(Q)

→
1 $x = Q[Q.head]$
2 **if** $Q.head == Q.size$
3 $Q.head = 1$
4 **else** $Q.head = Q.head + 1$
5 **return** x



↳ $T+1 = H$



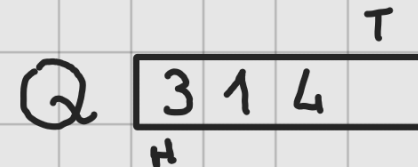
• ENQUEUE($Q, 3$)



• ENQUEUE($Q, 1$)



• ENQUEUE($Q, 4$)



FULL!!

↳ $H=1 \ \& \ T=Size$

~> ENQUEUE(Q, x):

$if (Q.HEAD = Q.TAIL + 1) \{$

$(Q.HEAD = 1 \ \& \ Q.TAIL = Q.SIZE):$

return "overflow"

...



EMPTY!!



↳ $Q.TAIL = Q.HEAD$

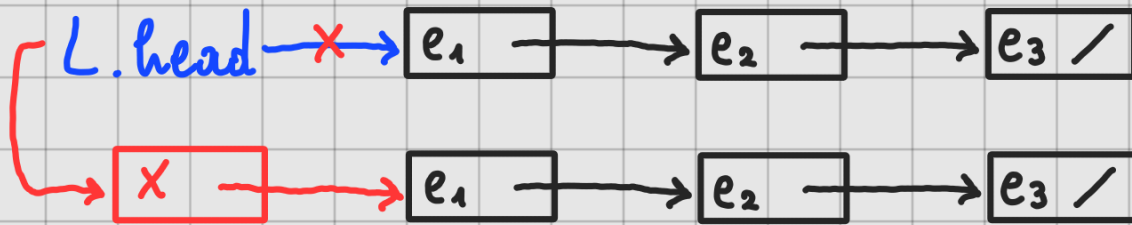
DEQUEUE(Q):

if $Q.TAIL = Q.HEAD$:

return "underflow"

...

Esercizio 2. L'operazione INSERT per gli insiemi dinamici può essere implementata per una lista singolarmente concatenata nel tempo $O(1)$? E l'operazione DELETE?



INSERT (L, x):

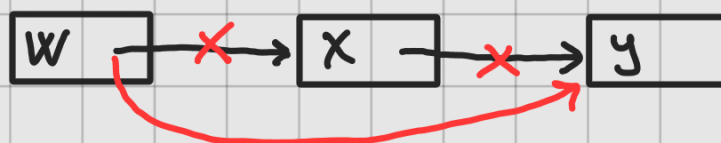
$x.next = L.head$

$L.head = x$

DELETE (L, x):

SCORRO LA LISTA: $\rightsquigarrow O(n)$

SE TROVO x :



Esercizio 3. Implementate uno stack utilizzando una lista singolarmente concatenata. Le operazioni PUSH e POP dovrebbero richiedere tempo $O(1)$.

PUSH

~~INSERT~~ (L, x):

$x.next = L.head$

$L.head = x$

POP (L):

if $L.head = NIL$:

return "underflow"

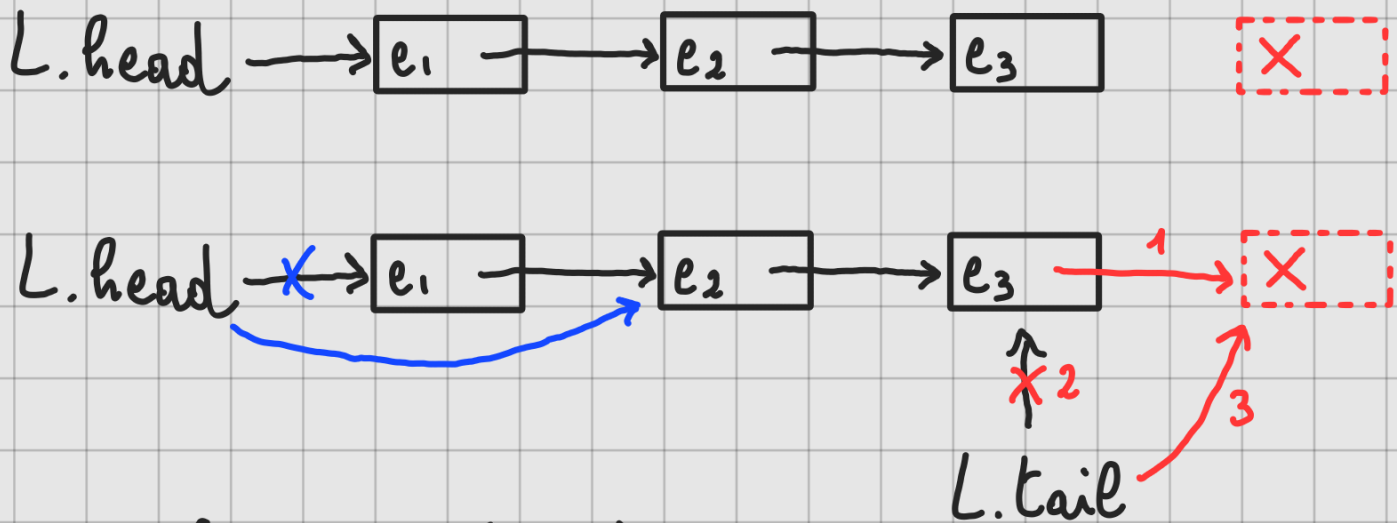
else:

$x = L.head$

$L.head = L.head.next$

return x

Esercizio 4. Implementate una coda utilizzando una lista singolarmente concatenata. Le operazioni ENQUEUE e DEQUEUE dovrebbero richiedere tempo $O(1)$.



ENQUEUE(L, x):

$L.tail.next = x$

$L.tail = x$

DEQUEUE(L):

$x = L.head$

$L.head = L.head.next$

return x

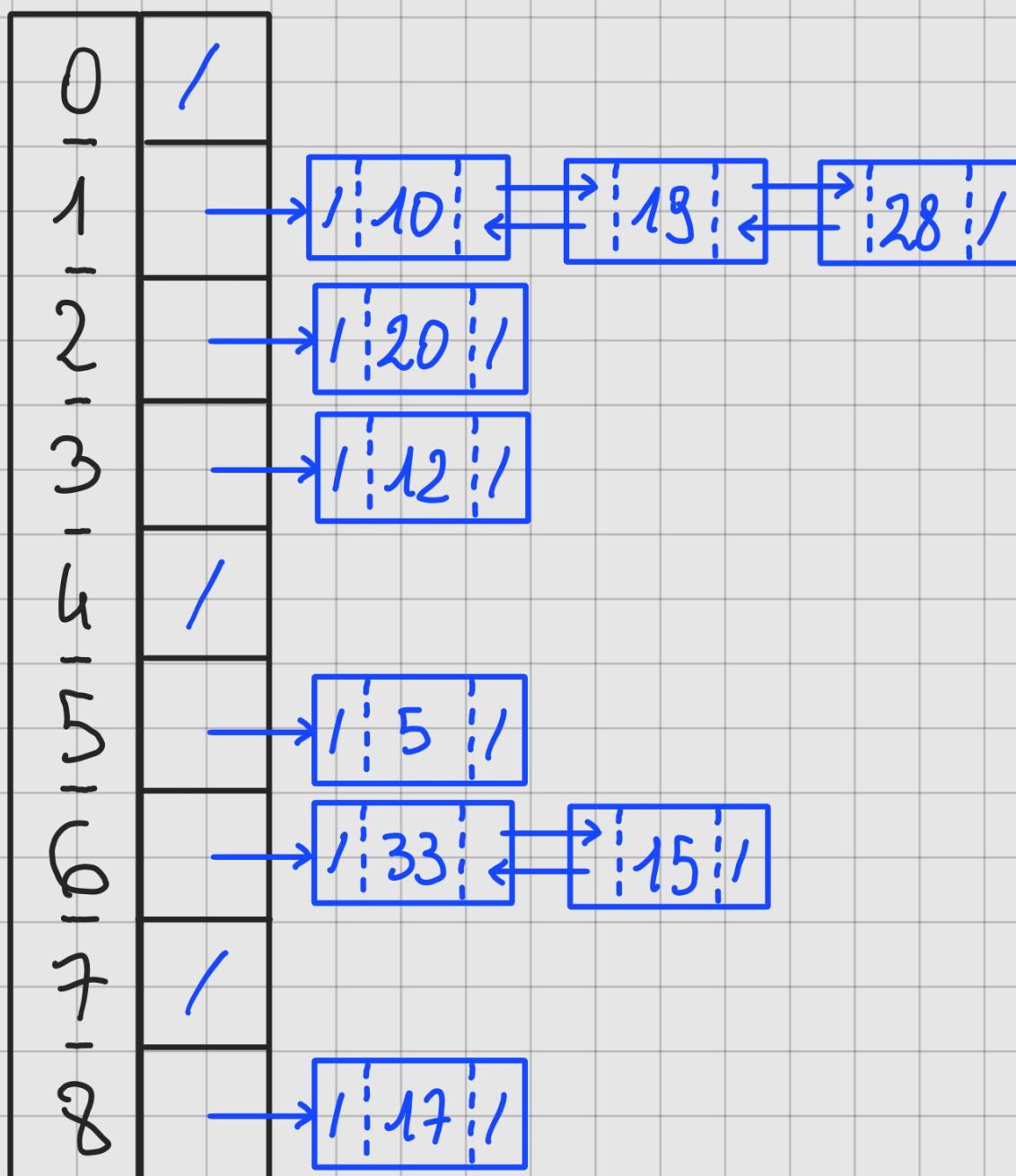
Esercizio 5. Considerate una tavola hash con 9 celle e la funzione hash $h(k) = k \bmod 9$. Partendo dalla tavola vuota, descrivete cosa succede quando si inseriscono in sequenza le chiavi

5, 28, 19, 15, 20, 33, 12, 17, 10,

risolvendo le collisioni mediante concatenamento.

$$h(k) = k \bmod 9$$

5, 28, 19, 15, 20, 33, 12, 17, 10
 $h \hookrightarrow$ 5, 1, 1, 6, 2, 6, 3, 8, 1



Esercizio 6. Dovete memorizzare un insieme di n chiavi in una tavola hash di dimensione m . Dimostrate che, se le chiavi appartengono ad un universo U con $|U| > (n-1)m$, allora esiste un sottoinsieme $U' \subseteq U$ di cardinalità n formato da chiavi che hanno tutte lo stesso valore di hash (i.e., esiste $i \in \{0, \dots, m-1\}$ tale che $h(k) = i$ per ogni $k \in U'$), e quindi il tempo di ricerca nel caso peggiore per l'hashing con concatenamento è $\Theta(n)$.

$$|H| = m \quad h \rightarrow \{0, \dots, m-1\} \quad |U| > (n-1)m$$

Claim: $\exists i \in \{0, \dots, m-1\} \wedge \exists U' \subseteq U, |U'| = n$
 t.c. $h(k) = i \quad \forall k \in U'$

Definiamo $U_j := \{k \in U : h(k) = j\} \quad \forall j \in \{0, \dots, m-1\}$

$$\hookrightarrow U = \bigcup_j (U_j) \quad \text{"l'unione su } j \text{"}$$

$$\hookrightarrow \forall j, j': j \neq j' \Rightarrow U_j \cap U_{j'} = \emptyset$$

$\hookrightarrow U_0, U_1, \dots, U_{m-1}$ è una partizione di U

$$\Rightarrow |U| = \sum_j |U_j| = |U_0| + |U_1| + \dots + |U_{m-1}|$$

Supponiamo per assurdo che $|U_j| < n \quad \forall j$ ↗ $\leq n-1$

$$\hookrightarrow \sum_j |U_j| \leq (n-1) \cdot m$$

$$\underbrace{\sum_j |U_j|}_{|U|} > (n-1) \cdot m \quad \text{ASSURDO!}$$

Quindi $\exists j$ t.c. $|U_j| \geq n$.

Prendiamo $U' \subseteq U_j$ t.c. $|U'| = n$ come richiesto.

Esercizio 7. Il professor Del Caso dichiara di aver scoperto un modo semplicissimo per definire una funzione hash che distribuisce uniformemente le chiavi in una tavola:

$$h(k) = \text{numero a caso in } \{0, \dots, m-1\}.$$

Una tavola hash basata su questa funzione randomizzata si comporterebbe correttamente?

INSERT: $\Theta(1)$

DELETE: $O(n)$ X

Per effettuare operazioni di ricerca, la fz di hash dato k deve restituire sempre lo stesso output.

↳ h deve essere DETERMINISTICA